

CHAPTER 8

MODERN ASYMMETRIC-KEY CIPHERS: PRINCIPLES



OUTLINES

1. Asymmetric-Key (Public-Key) Cryptography vs Symmetric-Key (Private-Key) Cryptography.
2. Principles in Asymmetric-Key Cryptography Design.
3. Trapdoor One-Way Function.



LEARNING OBJECTIVES

In the end of this Chapter 8, students will be able:

1. To understand and differentiate between **symmetric-key (private-key)** cryptography and **asymmetric-key (public-key)** cryptography.
2. To understand the **principles** in designing public-key cryptography and its application with respect to different security objectives.
3. To understand the **trapdoor one-way function** and its importance to public-key cryptography.



ASYMMETRIC-KEY CRYPTOGRAPHY **VS** **SYMMETRIC-KEY CRYPTOGRAPHY**



SYMMETRIC-KEY CRYPTOGRAPHY

- Recall that in symmetric-key cryptography, it **uses one private key** in encryption and decryption operations.
- This **same private key** (hence **symmetric**) is shared between the sender and the receiver. If this key is disclosed to the attacker, hence the cryptosystem is compromised.
- It has been implemented from ancient to modern times (including current time), in both the block and stream ciphers via substitution and/or permutation (transposition).
- This symmetric-key cryptography **DOES NOT** protect the **integrity** and **non-repudiation** properties in security.



ASYMMETRIC-KEY CRYPTOGRAPHY

- In contrast to symmetric-key cryptography, asymmetric-key cryptography uses **two different keys** (**one public key** and **one private key**, hence **asymmetric**) in encryption and decryption operations, respectively.
- Anyone can obtain such public key for encryption or verifying purposes, but can **NEVER** able to decrypt or sign the message without the corresponding private key.
- It is probably the most significant advancement in the 3000 years of history of cryptography through the **clever application of number theory**.
- It **complements rather than replaces** symmetric-key cryptography.



ASYMMETRIC-KEY CRYPTOGRAPHY:
BASIC CONCEPTS
&
DESIGN PRINCIPLES



ASYMMETRIC-KEY CRYPTOGRAPHY

- Why public-key cryptography is **important** (Why do we need it)?
 - It is developed to address **two key issues** associated with symmetric cryptography.
 1. **Key Distribution Problem**. To establish a secure communication in general without having to trust a Key Distribution Center.
 2. **Message Integrity**. To verify a message that comes intact from the claimed sender.
- The very first practical public-key cryptosystem was published by Whitfield Diffie and Martin Hellman in 1976, which realized the idea of **Key Distribution** (named the **Diffie-Hellman Key Exchange, DHKE**).



BASIC CONCEPTS

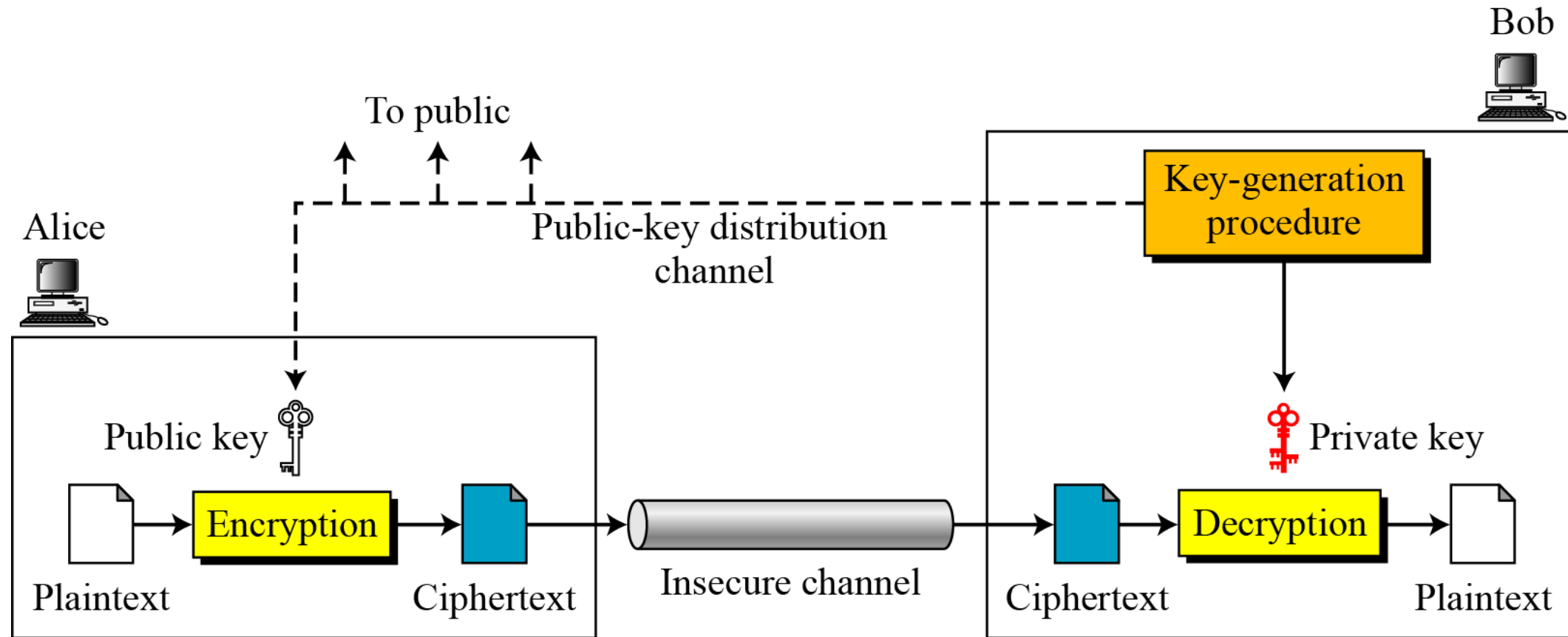
❖ As we have seen it many times, asymmetric-key cryptography uses two different keys:

1. **Public key** (K_{Pub}): to **encrypt** plaintext or to **verify** signature.
2. **Private key** (K_{Prv}) : to **decrypt** ciphertext or to **sign** message.

❖ In other words, once the plaintext is encrypted or a signature is produced, only the other user who possesses the counterpart can decrypt the ciphertext or verify the signature.



BASIC CONCEPTS



BASIC CONCEPTS

- ❖ Unlike symmetric-key cryptography that applies to **any forms of plaintext** (alphabets, bits, bytes, hexadecimal, etc.), the plaintext and ciphertext are treated as **integers** (decimals) in asymmetric-key cryptography.
- ❖ Hence, plaintext and ciphertext must be **encoded into integers** before encryption and decryption, respectively.



DESIGN PRINCIPLES

❖ The **design principles** of asymmetric-key cryptography rely on the following security objectives:

1. To provide **confidentiality**.

➤ The **key pair** is generated by **the receiver**, with private key known only to receiver while public key is known to everyone.

2. To provide **authentication**.

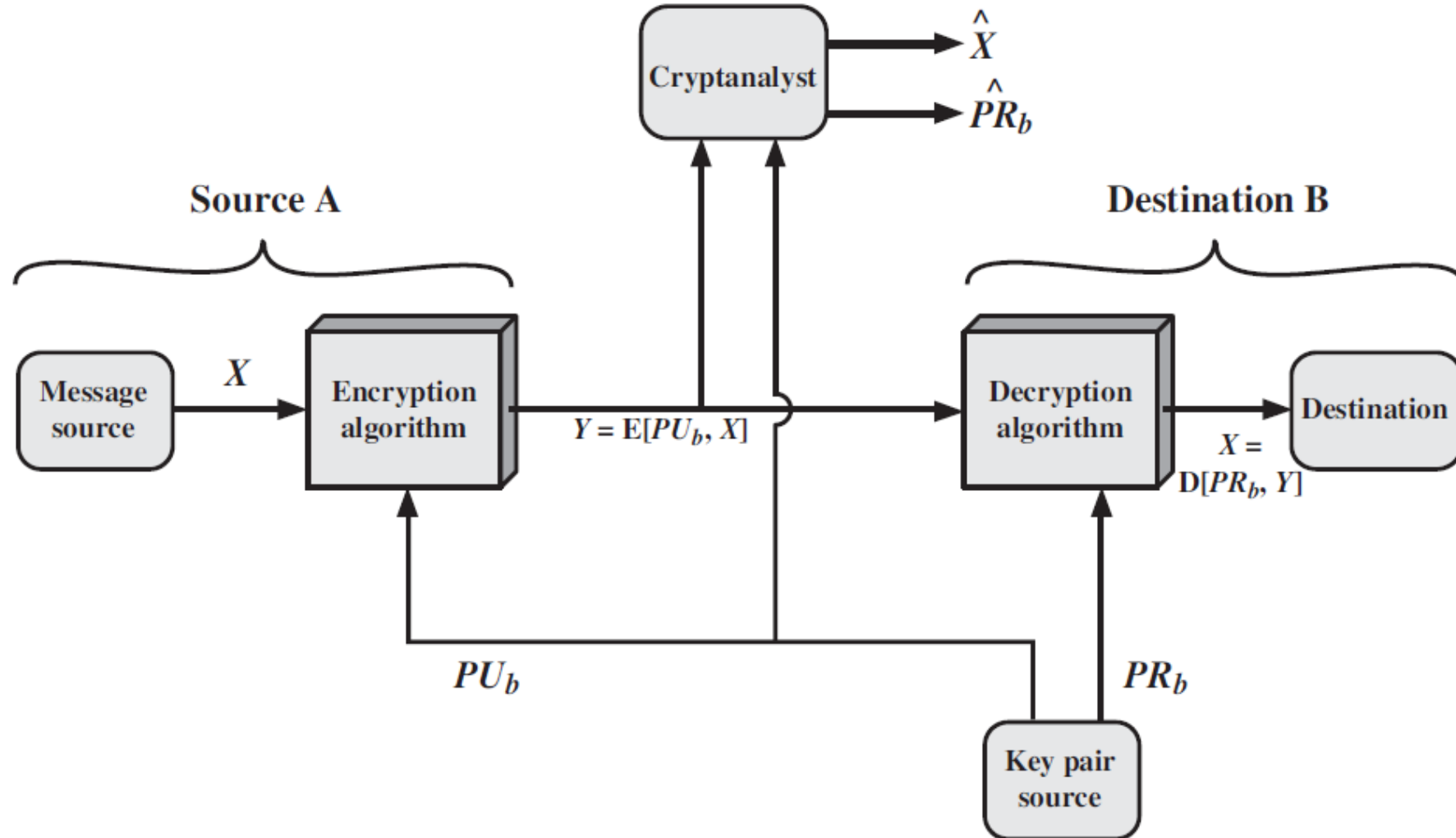
➤ The **key pair** is generated by **the sender**, with private key known only to sender while public key is known to everyone.

3. To provide both **confidentiality** and **authentication**.

➤ **Key pairs** are generated by both **sender** and **receiver**.



DESIGN PRINCIPLES – CONFIDENTIALITY

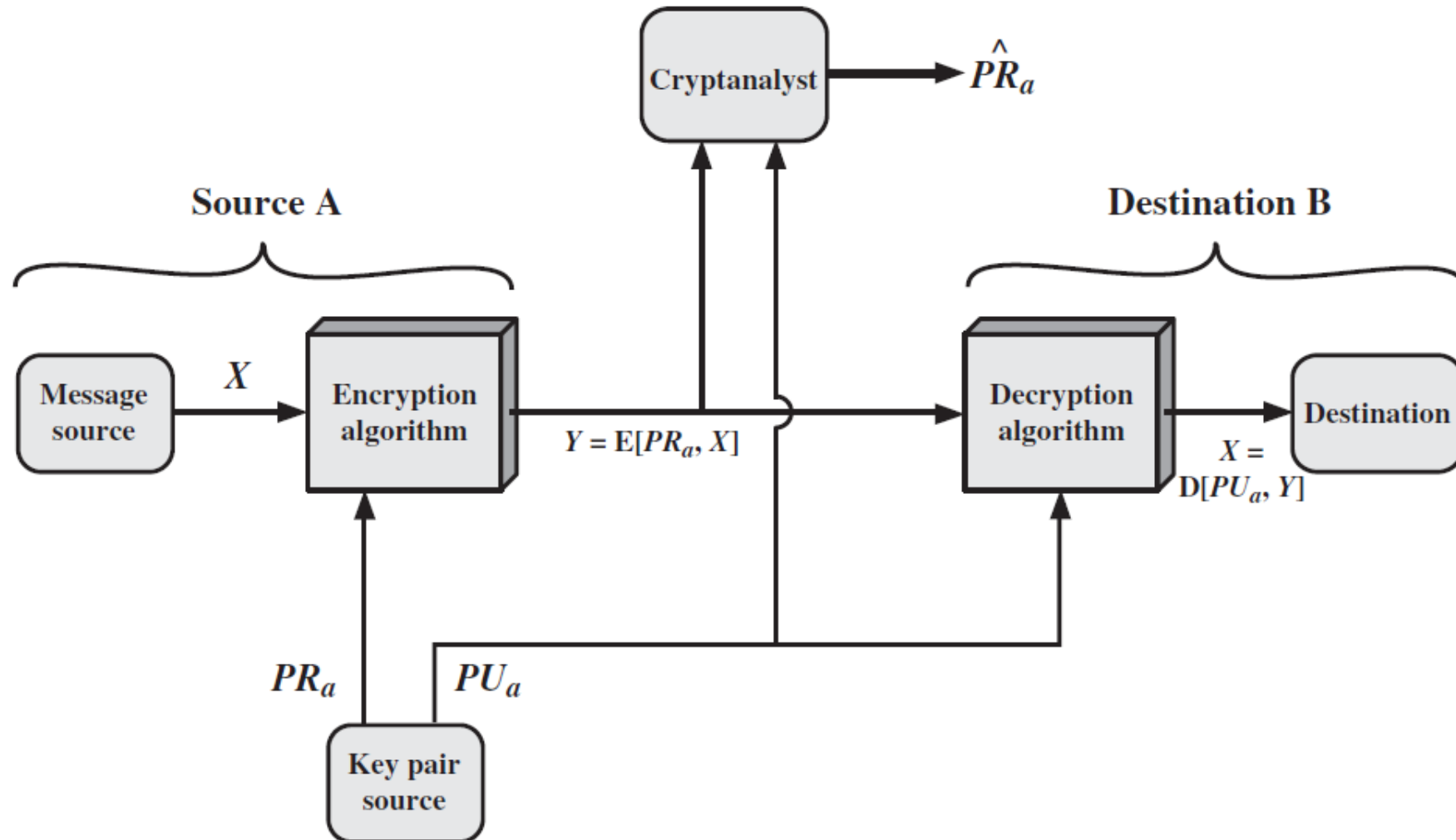


DESIGN PRINCIPLES — CONFIDENTIALITY

1. Source A produces a message in plaintext $X = [X_1, X_2, \dots, X_M]$. M elements of X are letters in some finite alphabet.
2. The message is intended for destination B.
3. B generates a related pair of keys: a public key PU_b (publicly available and accessible by A) and a private key, PR_b (only known to B).
4. With the message X and the encryption key PU_b , as input, A forms the ciphertext $Y = [Y_1, Y_2, \dots, Y_M]$, where $Y = \text{Enc}(PU_b, X)$.
5. The receiver, with the matching private key, can invert the transformation $X = \text{Dec}(PR_b, Y)$.



DESIGN PRINCIPLES — AUTHENTICATION

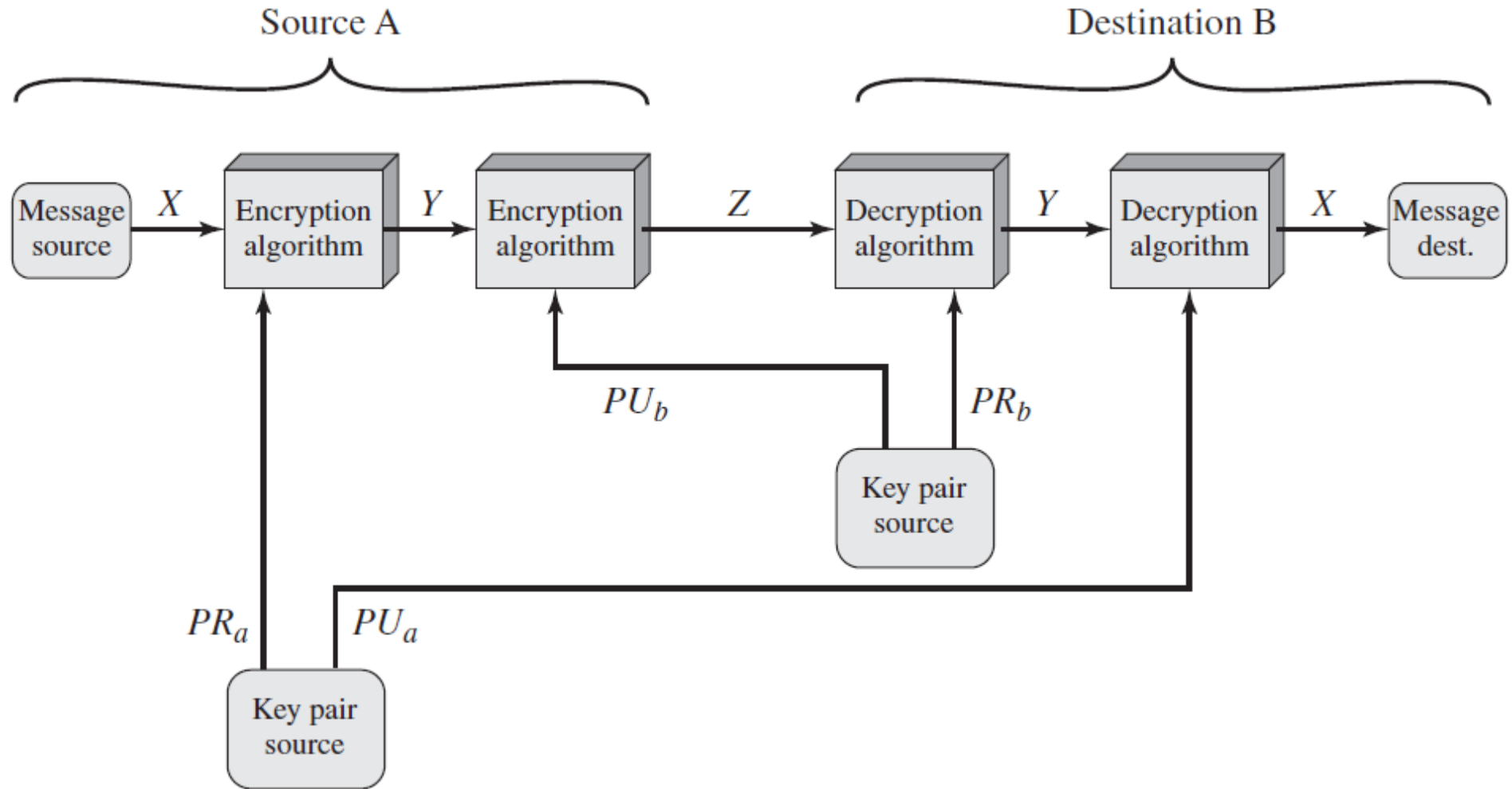


DESIGN PRINCIPLES — AUTHENTICATION

1. Source A prepares a message to B and encrypts it using A's private key before transmission.
2. B can decrypt the message using A's public key.
3. The entire encrypted message serves as a **digital signature**.
 - Message is authenticated in terms of source and in terms of data integrity.
 - Since the message was encrypted using A's private key, only A could have prepared the message and impossible to alter the message by others.
4. This **does not provide confidentiality** since any observer can decrypt the message by using the sender's public key.



DESIGN PRINCIPLES – CONFIDENTIALITY & AUTHENTICATION



DESIGN PRINCIPLES — CONFIDENTIALITY & AUTHENTICATION

1. Here the receiver owns and creates secrecy keys, sender owns and creates authentication keys.
2. Encryption of message done using sender's private key. This provides **authentication**.
3. This output is encrypted again using the receiver's public key to provide final ciphertext.
4. This final ciphertext can be decrypted only by the corresponding receiver, who has the matching private key. This provides **confidentiality**.
5. **Disadvantage**: The **computational cost is high** since the public-key algorithm must be executed four times.



APPLICATIONS

- There are **three** main applications of asymmetric-key cryptography:
 1. **Encryption/Decryption** (Confidentiality)
 - Sender encrypts a message with the recipient's public key.
 2. **Digital Signature** (Authentication)
 - The sender “signs” a message with its private key, either to the whole message or to a small block of data that is a function of the message.
 3. **Key Exchange**
 - Two sides cooperate to exchange a session key



APPLICATIONS

- Some algorithms are suitable for all the three applications, while others are specific to only one or two purposes.

Algorithm	Encryption/Decryption	Digital Signature	Key Exchange
RSA	Yes	Yes	Yes
Elliptic Curve	Yes	Yes	Yes
Diffie-Hellman	No	No	Yes
DSS	No	Yes	No

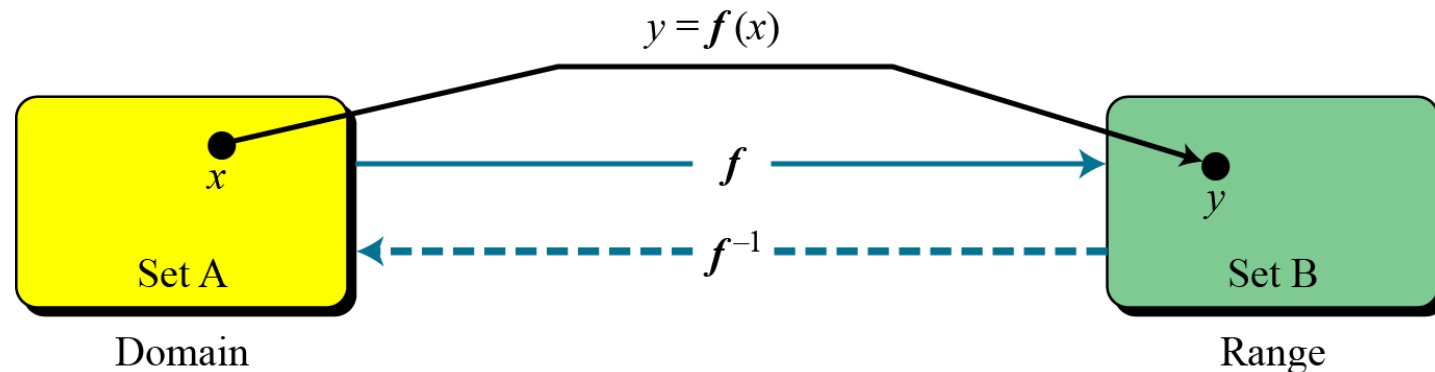


TRAPDOOR ONE-WAY FUNCTION



FUNCTION

- The underlying foundation of asymmetric-key cryptography is the concept of **trapdoor one-way function**.
- A **function** is a map on input of an element in the domain, outputs another element in the range.
- A function is **invertible** if it maps exactly one element in the range to one element in the domain.



ONE-WAY FUNCTION

- A **one-way function** is a function that maps an element in the domain into an element in the range such that:
 - **Every element has a unique inverse**
 - The **forward computation** of the function is **EASY**, but the backward **inversion** computation is **INFEASIBLE** (difficult).
- Mathematically, for a one-way function f_k :
 1. Given element x , computing $y = f_k(x)$ is easy.
 2. Given element y , inverting $x = f_k^{-1}(y)$ is computationally infeasible.



TRAPDOOR ONE-WAY FUNCTION

- **Trapdoor One-Way Function:**

- A one-way function that is given the third property, such that with **an additional information** (called **the trapdoor**), inverting the function is now feasible in polynomial time.

- Mathematically, for a one-way function f_k :

1. Given element x , computing $y = f_k(x)$ is easy.
2. Given element y , inverting $x = f_k^{-1}(y)$ is computationally infeasible.
3. Given element y and **the trapdoor k** , inverting $x = f_k^{-1}(y)$ is easy.



TRAPDOOR ONE-WAY FUNCTION

- Some famous examples of o trapdoor one-way functions that we used in cryptography (and we will learn it in the next chapter):
 1. **Integer Factorization Problem (IFP)**. The core security of the RSA cryptosystem.
 2. **Discrete Logarithm Problem (DLP)**. The fundamental strength of the Diffie-Hellman Key Exchange, and the ElGamal cryptosystem.
 3. **Knapsack Problem** (A family of **Subset Sum Problem (SSP)**). The heart of the Knapsack cryptosystem.



MAIN REFERENCES FOR CHAPTER 8

1. Forouzan, B.A. “*Introduction to Cryptography and Network Security*”, Third Edition, McGraw-Hill, 2008.
2. Stallings, W. “*Cryptography and Network Security: Principles and Practices*”, Fifth Edition, Pearson Prentice Hall, 2011.



~ THE END 😊 ~

